

Backpropagation Through a Network

Deep Learning

Daniel E. Acuna

Associate Professor, University of Colorado Boulder

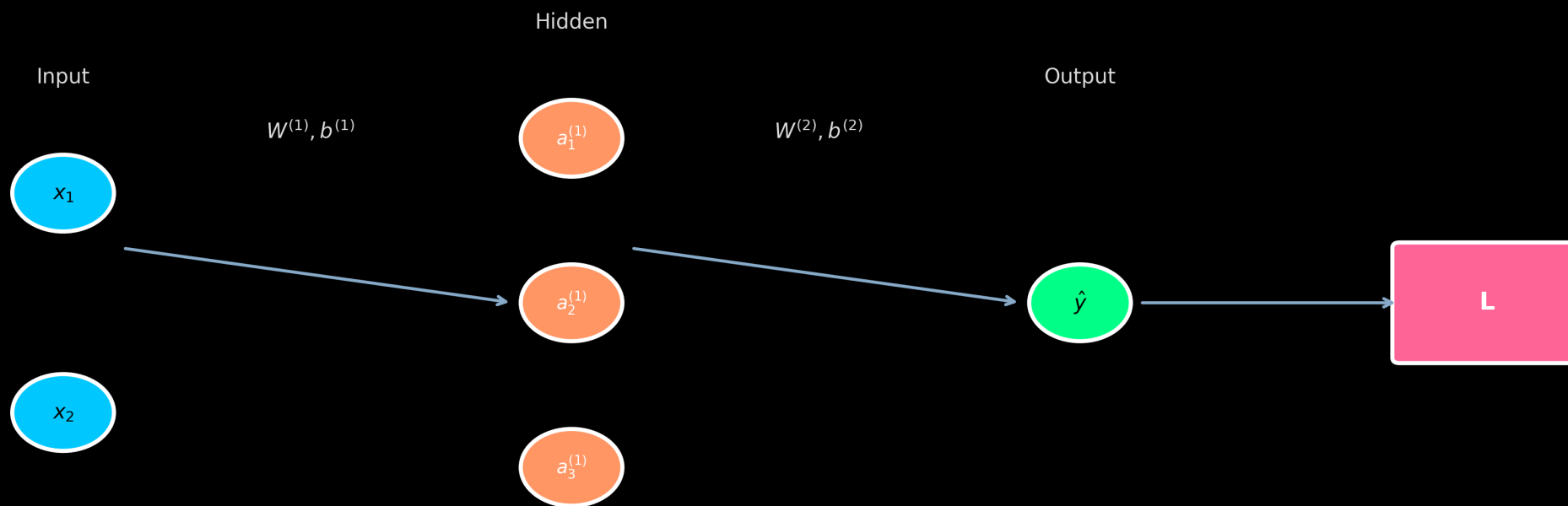
Contents of This Video

In this video, we will cover:

- Extending backpropagation to multiple layers
- The backward pass algorithm
- How gradients propagate through hidden layers
- A two-layer network example
- Automatic differentiation in deep learning frameworks

A Two-Layer Network

Two-Layer Network: Input $\rightarrow$ Hidden $\rightarrow$ Output $\rightarrow$ Loss



The Forward Pass (Review)

Layer 1:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)})$$

Layer 2:

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

$$\hat{y} = \sigma(\mathbf{z}^{(2)})$$

Loss:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

The Backward Pass: Start from Loss

Step 1: Gradient at output

$$\frac{\partial L}{\partial z^{(2)}} = \hat{y} - y$$

(This combines $\frac{\partial L}{\partial \hat{y}}$ and $\frac{\partial \hat{y}}{\partial z^{(2)}}$)

Step 2: Gradients for Layer 2 parameters

$$\frac{\partial L}{\partial W^{(2)}} = \frac{\partial L}{\partial z^{(2)}} \cdot (\mathbf{a}^{(1)})^\top$$

$$\frac{\partial L}{\partial b^{(2)}} = \frac{\partial L}{\partial z^{(2)}}$$

The Backward Pass: Propagate to Hidden Layer

Step 3: Propagate gradient to hidden layer

$$\frac{\partial L}{\partial \mathbf{a}^{(1)}} = (W^{(2)})^\top \frac{\partial L}{\partial \mathbf{z}^{(2)}}$$

Step 4: Gradient through ReLU

$$\frac{\partial L}{\partial \mathbf{z}^{(1)}} = \frac{\partial L}{\partial \mathbf{a}^{(1)}} \odot \mathbb{1}[\mathbf{z}^{(1)} > 0]$$

($\odot$ = element-wise multiply, $\mathbb{1}[\cdot]$ = indicator function)

The Backward Pass: Layer 1 Parameters

Step 5: Gradients for Layer 1 parameters

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \frac{\partial L}{\partial \mathbf{z}^{(1)}} \cdot \mathbf{x}^\top$$

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \frac{\partial L}{\partial \mathbf{z}^{(1)}}$$

Same pattern as Layer 2!

- Weight gradient = output gradient $\times$ input to layer
- Bias gradient = output gradient

The Complete Backpropagation Algorithm

Forward Pass: (compute and store)

1. For each layer $l = 1, 2, \dots, L$:

- $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$
- $\mathbf{a}^{(l)} = \sigma_l(\mathbf{z}^{(l)})$

2. Compute loss L

Backward Pass: (compute gradients)

3. $\delta^{(L)} = \frac{\partial L}{\partial \mathbf{z}^{(L)}}$

4. For each layer $l = L, L - 1, \dots, 1$:

- $\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$
- $\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$
- $\delta^{(l-1)} = (\mathbf{W}^{(l)})^\top \delta^{(l)} \odot \sigma'_{l-1}(\mathbf{z}^{(l-1)})$

Automatic Differentiation

In practice, we don't implement backprop by hand!

Modern deep learning tools provide **automatic differentiation**:

1. You define the network architecture
2. The framework builds a computation graph during forward pass
3. It automatically computes all gradients via backpropagation
4. Weight updates are handled for you

This automation is one of the most important innovations in deep learning

Why Understanding Backprop Matters

Even though frameworks handle it automatically:

Debugging: Shape mismatches, vanishing gradients, NaN losses

Design: Which architectures will train well?

Intuition: Why do skip connections help? Why does batch norm work?

Efficiency: Memory usage during training (storing activations)

What We've Covered

In this video, we've learned:

- Backpropagation applies the chain rule layer by layer
- Forward pass: compute and store activations
- Backward pass: propagate δ from output to input
- Weight gradient = $\delta \cdot (\text{input to layer})^\top$
- Automatic differentiation handles backprop in practice
- Understanding helps with debugging and architecture design